

Singular Value Decomposition and PCA from Scratch

Vatsav Reddy

Under the Guidance of
Prof. Praveen Parchuri

International Institute of Information Technology, Hyderabad

June 7, 2026

Contents

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

Variance: Measuring Spread of a Single Variable

Population Variance

For a random variable x with mean $\mu_x = \mathbb{E}[x]$, the **variance** measures how far values typically deviate from the mean:

$$\text{Var}(x) = \mathbb{E}[(x - \mu_x)^2] = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$$

The second equality follows by expanding $(x - \mu_x)^2$ and using linearity of expectation:
 $\mathbb{E}[(x - \mu_x)^2] = \mathbb{E}[x^2] - 2\mu_x\mathbb{E}[x] + \mu_x^2 = \mathbb{E}[x^2] - \mu_x^2.$

Sample Variance (from data)

Given n observations $x_1, x_2, \dots, x_n$ with sample mean $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

The factor $n-1$ (Bessel's correction) makes s^2 an unbiased estimator of the true population variance. In machine learning we often use $\frac{1}{n}$ for convenience.

Intuition

Large variance $\Rightarrow$ data is spread out; small variance $\Rightarrow$ data is tightly clustered near the mean. PCA will later exploit variance to identify *informative* directions.

Covariance: How Two Variables Move Together

Population Covariance

For two random variables x and y with means μ_x, μ_y :

$$\text{Cov}(x, y) = \mathbb{E}[(x - \mu_x)(y - \mu_y)] = \mathbb{E}[xy] - \mathbb{E}[x]\mathbb{E}[y]$$

This measures *joint* variability.

Sample Covariance

$$\widehat{\text{Cov}}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- **Positive** $\Rightarrow x$ and y tend to increase *together* (positive association)
- **Negative** $\Rightarrow$ one increases while the other decreases (opposite movement)
- **Near zero** $\Rightarrow$ no linear relationship between x and y

Note: $\text{Cov}(x, x) = \text{Var}(x)$ — covariance with itself is variance.

Correlation: Normalised Covariance

Pearson Correlation Coefficient

Raw covariance depends on the *units* of x and y . Normalise by the product of standard deviations to get a unit-free measure:

$$\rho_{xy} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}, \quad \sigma_x = \sqrt{\text{Var}(x)}, \quad \sigma_y = \sqrt{\text{Var}(y)}$$

$$-1 \leq \rho_{xy} \leq 1$$

Sample version:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Interpretation

$\rho = +1$: perfect positive linear relationship. $\rho = 0$: no linear relationship. $\rho = -1$: perfect negative linear relationship. High $|\rho|$ implies one variable is approximately a linear function of the other — one is *redundant*. PCA eliminates this redundancy.

The Covariance Matrix

Setup: Multivariate Data

Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$ be a random vector. The **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$ is:

$$\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top], \quad \boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$$

Entry (i, j) of Σ is $\Sigma_{ij} = \text{Cov}(x_i, x_j)$.

Explicit 2×2 Example

For $\mathbf{x} = [x, y]^\top$:

$$\Sigma = \begin{bmatrix} \text{Var}(x) & \text{Cov}(x, y) \\ \text{Cov}(y, x) & \text{Var}(y) \end{bmatrix}$$

The matrix is **symmetric** ($\Sigma_{ij} = \Sigma_{ji}$) because $\text{Cov}(x_i, x_j) = \text{Cov}(x_j, x_i)$.

Correlation Matrix

Normalise each entry by $\sigma_i \sigma_j$: $R_{ij} = \frac{\text{Cov}(x_i, x_j)}{\sigma_i \sigma_j}$, so $R_{ii} = 1$ (diagonal is all ones).

Sample Covariance Matrix from Data

Data Matrix and Sample Covariance

With m data points in $\mathbb{R}^n$, arrange them as rows of $X \in \mathbb{R}^{m \times n}$. After centring (zero-mean columns): $X_c = X - \mathbf{1}\boldsymbol{\mu}^\top$ where $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i$.

The **sample covariance matrix** is:

$$\Sigma = \frac{1}{m} X_c^\top X_c \in \mathbb{R}^{n \times n}$$

Entry (j, k) : $\Sigma_{jk} = \frac{1}{m} \sum_{i=1}^m (X_c)_{ij} (X_c)_{ik} =$ sample covariance between feature j and feature k .

Two Key Properties

- 1 **Symmetric**: $\Sigma^\top = \frac{1}{m} (X_c^\top X_c)^\top = \frac{1}{m} X_c^\top X_c = \Sigma$. Follows from $(AB)^\top = B^\top A^\top$.
- 2 **Positive semi-definite (PSD)**: For any $\mathbf{v} \in \mathbb{R}^n$,
 $\mathbf{v}^\top \Sigma \mathbf{v} = \frac{1}{m} \mathbf{v}^\top X_c^\top X_c \mathbf{v} = \frac{1}{m} \|X_c \mathbf{v}\|^2 \geq 0$. Hence all eigenvalues $\lambda_i \geq 0$.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

Eigenvectors and Eigenvalues: Recap

Definition

For a square matrix $A \in \mathbb{R}^{n \times n}$, a nonzero vector $\mathbf{v}$ is an **eigenvector** with **eigenvalue** λ if:

$$A\mathbf{v} = \lambda\mathbf{v}$$

A stretches or flips $\mathbf{v}$ along its own direction — *no rotation*, only scaling. To find λ : rearrange as $(A - \lambda I)\mathbf{v} = \mathbf{0}$. For a nonzero solution $\mathbf{v}$ to exist, the matrix $(A - \lambda I)$ must be singular, i.e. $\det(A - \lambda I) = 0$. This is the *characteristic polynomial* of A .

Why Eigenvectors Matter: Matrix Multiplication Becomes Scalar

If $\mathbf{x} = \sum_{i=1}^n \alpha_i \mathbf{v}_i$ (eigenvectors as basis), then:

$$A\mathbf{x} = A \sum_{i=1}^n \alpha_i \mathbf{v}_i = \sum_{i=1}^n \alpha_i A\mathbf{v}_i = \sum_{i=1}^n \alpha_i \lambda_i \mathbf{v}_i$$

The matrix multiplication reduces to **scalar multiplications** $\alpha_i \mapsto \alpha_i \lambda_i$. This is the fundamental computational advantage of the eigenbasis.

Eigenvalue Decomposition (EVD)

Construction

Suppose A has n linearly independent eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ with eigenvalues $\lambda_1, \dots, \lambda_n$. Stack eigenvectors as columns of $V \in \mathbb{R}^{n \times n}$:

$$AV = A[\mathbf{v}_1 \ \dots \ \mathbf{v}_n] = [\lambda_1 \mathbf{v}_1 \ \dots \ \lambda_n \mathbf{v}_n] = V\Lambda$$

where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$. If V is invertible (eigenvectors linearly independent):

$$\boxed{A = V\Lambda V^{-1}} \quad \Longleftrightarrow \quad \Lambda = V^{-1}AV$$

Spectral Theorem for Symmetric Matrices

If A is **real symmetric** ($A = A^\top$), then:

- 1 All eigenvalues are **real**.
- 2 Eigenvectors corresponding to *distinct* eigenvalues are **orthogonal**.
- 3 A has a complete orthonormal set of eigenvectors, so V can be chosen to satisfy $V^\top V = I$, meaning $V^{-1} = V^\top$.

Therefore, for a real symmetric A :

$$\boxed{A = V\Lambda V^\top}$$

This is the **Spectral Decomposition**.

Applying EVD to the Covariance Matrix

$$\text{Diagonalising } \Sigma = \frac{1}{m} X_c^\top X_c$$

Since Σ is real symmetric and PSD, the Spectral Theorem gives:

$$\Sigma \mathbf{p}_i = \lambda_i \mathbf{p}_i, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0, \quad \mathbf{p}_i^\top \mathbf{p}_j = \delta_{ij}$$

Collecting into matrices:

$$\Sigma = P \Lambda P^\top, \quad P = [\mathbf{p}_1 \dots \mathbf{p}_n], \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$$

Key insight: $\mathbf{p}_i^\top \Sigma \mathbf{p}_i = \lambda_i$, so the variance of the projected data along direction $\mathbf{p}_i$ equals λ_i . Large eigenvalue = high variance direction.

Computational Note

In practice, use `numpy.linalg.eigh` (not `eig`) for symmetric matrices: it returns *real* eigenvalues in ascending order and is more numerically stable. Reverse to get descending order (largest first).

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost**
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

The Computational Problem

Why Direct EVD of Σ is Expensive

For face images of size 100×100 , we have $n = 10,000$ features (pixels). The covariance matrix $\Sigma = \frac{1}{m} X_c^\top X_c$ is then $n \times n = 10,000 \times 10,000$.

- **Memory:** $n^2 = 10^8$ floating point entries ≈ 800 MB
- **Eigendecomposition:** $O(n^3) = O(10^{12})$ operations — impractical

The Gram Matrix Alternative

Instead of diagonalising the $n \times n$ matrix Σ , diagonalise the much smaller $m \times m$ **Gram matrix**:

$$G = \frac{1}{m} X_c X_c^\top \in \mathbb{R}^{m \times m}$$

With $m = 400$ images and $n = 10,000$ pixels, this gives an EVD cost of $O(m^3) = O(400^3) = O(6.4 \times 10^7)$ operations instead of $O(10^{12})$ — a speedup of $(\frac{n}{m})^3 = 25^3 = 15,625\times$.

Why G and Σ Share the Same Non-Zero Eigenvalues

Shared Spectrum Identity

Let $G\mathbf{v}_i = \lambda_i\mathbf{v}_i$ for some nonzero $\mathbf{v}_i \in \mathbb{R}^m$ and $\lambda_i \neq 0$. We claim $\mathbf{p}_i := X_c^\top \mathbf{v}_i$ is an eigenvector of Σ with the *same* eigenvalue λ_i .

Proof: Expand $G\mathbf{v}_i = \lambda_i\mathbf{v}_i$:

$$\frac{1}{m} X_c X_c^\top \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

Multiply both sides on the left by $X_c^\top$:

$$X_c^\top \left(\frac{1}{m} X_c X_c^\top \mathbf{v}_i \right) = X_c^\top (\lambda_i \mathbf{v}_i)$$

$$\underbrace{\frac{1}{m} X_c^\top X_c}_{\Sigma} \underbrace{X_c^\top \mathbf{v}_i}_{\mathbf{p}_i} = \lambda_i \underbrace{X_c^\top \mathbf{v}_i}_{\mathbf{p}_i}$$

$$\therefore \boxed{\Sigma \mathbf{p}_i = \lambda_i \mathbf{p}_i}$$

$\mathbf{p}_i = X_c^\top \mathbf{v}_i$ is indeed an eigenvector of Σ with the *same* eigenvalue λ_i . □

Normalising the Pixel-Space Eigenvector

Deriving the Normalisation Factor $\sqrt{m\lambda_i}$

We have $\mathbf{p}_i \propto X_c^\top \mathbf{v}_i$. To make $\|\mathbf{p}_i\| = 1$, compute the norm:

$$\|X_c^\top \mathbf{v}_i\|^2 = (X_c^\top \mathbf{v}_i)^\top (X_c^\top \mathbf{v}_i) = \mathbf{v}_i^\top X_c X_c^\top \mathbf{v}_i$$

Since $X_c X_c^\top = mG$ (by definition), and $G\mathbf{v}_i = \lambda_i \mathbf{v}_i$:

$$= \mathbf{v}_i^\top (mG) \mathbf{v}_i = m \mathbf{v}_i^\top G \mathbf{v}_i = m \mathbf{v}_i^\top (\lambda_i \mathbf{v}_i) = m \lambda_i \underbrace{\|\mathbf{v}_i\|^2}_{=1} = m \lambda_i$$

Therefore $\|X_c^\top \mathbf{v}_i\| = \sqrt{m\lambda_i}$, and the **normalised pixel-space eigenvector** (eigenface) is:

$$\boxed{\mathbf{p}_i = \frac{X_c^\top \mathbf{v}_i}{\sqrt{m\lambda_i}}}, \quad \|\mathbf{p}_i\| = 1$$

Interpretation

$\mathbf{v}_i \in \mathbb{R}^m$: coefficients telling how to linearly combine the m training images. $X_c^\top \mathbf{v}_i$: the resulting pixel pattern from that combination. After normalisation: unit-norm direction in pixel space.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline**
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

PCA: Step-by-Step Algorithm

Step 1 — Centre the Data

Compute the mean image: $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i \in \mathbb{R}^n$. Centre: $X_c = X - \mathbf{1}_m \boldsymbol{\mu}^\top$. Now every column of X_c has zero mean. *Why?* Covariance measures spread *around the mean*; centring ensures $\Sigma = \frac{1}{m} X_c^\top X_c$ correctly represents variances and covariances.

Step 2 — Eigendecompose (via Gram Matrix if $m \ll n$)

Compute $G = \frac{1}{m} X_c X_c^\top$ and solve $G \mathbf{v}_i = \lambda_i \mathbf{v}_i$. Recover pixel-space eigenvectors via $\mathbf{p}_i = \frac{X_c^\top \mathbf{v}_i}{\sqrt{m \lambda_i}}$. Order by $\lambda_1 \geq \lambda_2 \geq \dots$.

Step 3 — Select Top- k Principal Components

Form $P_k = [\mathbf{p}_1, \dots, \mathbf{p}_k]^\top \in \mathbb{R}^{k \times n}$ (each row is a principal direction). These k directions capture the most variance.

Step 4 — Project

$$Z = X_c P_k^\top \in \mathbb{R}^{m \times k}$$

Each row $\mathbf{z}_i = X_{c,i} P_k^\top \in \mathbb{R}^k$ gives the PCA coefficients (scores) for data point i . The j -th score $z_{ij} = \mathbf{x}_{c,i}^\top \mathbf{p}_j$: the projection of image i onto eigenface j .

Reconstruction and Reconstruction Error

Reconstruction Formula

Recover the original (approximate) image from PCA codes:

$$\hat{\mathbf{x}}_i = \boldsymbol{\mu} + \mathbf{z}_i \mathbf{P}_k = \boldsymbol{\mu} + \mathbf{x}_{c,i} \mathbf{P}_k^\top \mathbf{P}_k$$

In summation form (most interpretable):

$$\hat{\mathbf{x}}_i = \boldsymbol{\mu} + \sum_{j=1}^k \underbrace{(\mathbf{x}_{c,i}^\top \mathbf{p}_j)}_{z_{ij}} \mathbf{p}_j$$

Reading this: Start from the mean face $\boldsymbol{\mu}$. Compute how much of each eigenface $\mathbf{p}_j$ is present (the scalar z_{ij}). Add those weighted eigenfaces back.

Why $\mathbf{P}_k^\top \mathbf{P}_k \neq \mathbf{I}$

$\mathbf{P}_k \in \mathbb{R}^{k \times n}$ has orthonormal *rows*, so $\mathbf{P}_k \mathbf{P}_k^\top = \mathbf{I}_k$ (rows are unit, orthogonal). But $\mathbf{P}_k^\top \mathbf{P}_k \in \mathbb{R}^{n \times n}$ is a rank- k orthogonal projector onto the subspace spanned by $\mathbf{p}_1, \dots, \mathbf{p}_k$, satisfying $(\mathbf{P}_k^\top \mathbf{P}_k)^2 = \mathbf{P}_k^\top \mathbf{P}_k$. The reconstruction projects $\mathbf{x}_{c,i}$ onto this k -dimensional subspace.

Explained Variance Ratio

How Much Information Does Each Component Capture?

The variance of the projected data along direction $\mathbf{p}_i$:

$$\frac{1}{m} \|\hat{X}_i\|^2 = \frac{1}{m} (X_c \mathbf{p}_i)^\top (X_c \mathbf{p}_i) = \frac{1}{m} \mathbf{p}_i^\top X_c^\top X_c \mathbf{p}_i = \mathbf{p}_i^\top \Sigma \mathbf{p}_i = \mathbf{p}_i^\top (\lambda_i \mathbf{p}_i) = \lambda_i$$

So the variance along the i -th principal component is exactly λ_i (the corresponding eigenvalue).

The **Explained Variance Ratio** (EVR) for component i :

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^n \lambda_j}$$

This is the fraction of *total variance* captured by direction $\mathbf{p}_i$. To choose k : find the smallest k such that $\sum_{i=1}^k \text{EVR}_i$ exceeds a threshold (e.g. 0.95).

Practical Advice (from lecture)

Set aside 100 validation images. Compute reconstruction error for $k = 100, 1000, 10,000$. The right k is application-dependent: medical/industrial images need much lower reconstruction error than coarse face recognition.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)**
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

The Problem with Rectangular Matrices

Eigenvectors Require Square Matrices

The eigenvalue equation $A\mathbf{v} = \lambda\mathbf{v}$ requires both sides to live in the *same* space $\mathbb{R}^n$. For a **rectangular** matrix $A \in \mathbb{R}^{m \times n}$ with $m \neq n$:

$$\underbrace{A}_{\in \mathbb{R}^n} \underbrace{\mathbf{v}}_{\in \mathbb{R}^n} = \underbrace{\mathbf{w}}_{\in \mathbb{R}^m}$$

The output lives in a *different* space than the input. The equation $A\mathbf{v} = \lambda\mathbf{v}$ cannot hold because $\lambda\mathbf{v} \in \mathbb{R}^n$ while $A\mathbf{v} \in \mathbb{R}^m$. Hence rectangular matrices **cannot have eigenvectors**.

Why This Is a Real Problem

Data matrices $X \in \mathbb{R}^{m \times n}$ are almost always rectangular. We *want* the computational advantage of eigenvectors (converting matrix multiplication to scalar multiplication), but we cannot directly apply EVD. SVD is the resolution: it finds *two* sets of orthonormal vectors — one for the input space $\mathbb{R}^n$ and one for the output space $\mathbb{R}^m$ — that *together* make the matrix operation scalar.

SVD: The Core Idea

Looking for Paired Bases in Two Different Spaces

Suppose we can find:

- $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$ (orthonormal, forming a basis for the *row space* of A)
- $\mathbf{u}_1, \dots, \mathbf{u}_k \in \mathbb{R}^m$ (orthonormal, forming a basis for the *column space* of A)

such that:

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad i = 1, \dots, k$$

where $\sigma_i > 0$ are scalars called **singular values**.

If every $\mathbf{x} \in \mathbb{R}^n$ is written as $\mathbf{x} = \sum_{i=1}^k \alpha_i \mathbf{v}_i$, then:

$$A\mathbf{x} = \sum_{i=1}^k \alpha_i A\mathbf{v}_i = \sum_{i=1}^k \alpha_i \sigma_i \mathbf{u}_i$$

Once again, **matrix multiplication reduces to scalar multiplication** — this is the exact same advantage as eigenvectors, but now valid for rectangular matrices.

SVD: Matrix Form

Writing $A\mathbf{v}_i = \sigma_i\mathbf{u}_i$ as a Matrix Equation

Stack all relations simultaneously. Let $V = [\mathbf{v}_1 \cdots \mathbf{v}_n] \in \mathbb{R}^{n \times n}$ (all right singular vectors, extended to a full basis of $\mathbb{R}^n$ via Gram–Schmidt for the null space directions). Let $U = [\mathbf{u}_1 \cdots \mathbf{u}_m] \in \mathbb{R}^{m \times m}$ (left singular vectors, similarly extended). Let $\Sigma \in \mathbb{R}^{m \times n}$ be the matrix with $\Sigma_{ij} = \sigma_i$ for $i = 1, \dots, k$ and zeros elsewhere. Then:

$$AV = U\Sigma \quad \Rightarrow \quad \boxed{A = U\Sigma V^T}$$

This is the **Singular Value Decomposition** of A . Since U and V are orthogonal ($UU^T = I$, $VV^T = I$), their inverses are their transposes.

Dimensions

$U \in \mathbb{R}^{m \times m}$ (left singular vectors), $\Sigma \in \mathbb{R}^{m \times n}$ (diagonal, singular values), $V \in \mathbb{R}^{n \times n}$ (right singular vectors). The *thin* (economy) SVD keeps only the $k = \text{rank}(A)$ non-zero singular values: $U_k \in \mathbb{R}^{m \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$, $V_k \in \mathbb{R}^{n \times k}$.

How to Find U and V : Connection to EVD

Computing V (Right Singular Vectors)

Suppose $A = U\Sigma V^T$ exists. Compute $A^T A$:

$$A^T A = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T U^T U\Sigma V^T = V \underbrace{\Sigma^T \Sigma}_{\text{diagonal: } \sigma_i^2} V^T$$

(using $U^T U = I$ since U is orthogonal). This is exactly the **EVD of $A^T A$** : $A^T A = V\Lambda_V V^T$ where $\Lambda_V = \Sigma^T \Sigma$ has entries $\lambda_i = \sigma_i^2$. Therefore: **V consists of the eigenvectors of $A^T A$.**

Computing U (Left Singular Vectors)

Similarly, compute AA^T :

$$AA^T = (U\Sigma V^T)(U\Sigma V^T)^T = U\Sigma V^T V\Sigma^T U^T = U \underbrace{\Sigma \Sigma^T}_{\text{diagonal: } \sigma_i^2} U^T$$

This is the EVD of AA^T : $AA^T = U\Lambda_U U^T$. Therefore: **U consists of the eigenvectors of AA^T .** And the singular values $\sigma_i = \sqrt{\lambda_i}$ (square roots of the common eigenvalues of $A^T A$ and AA^T).

Why SVD Always Exists

SVD vs. EVD: Existence Conditions

EVD of A requires:

- 1 A is **square** ($n \times n$)
- 2 A has n linearly independent eigenvectors (not always guaranteed)

SVD has **no such restrictions**. It always exists because:

- $A^T A$ is always a **real symmetric PSD** matrix (regardless of the shape of A). Symmetric matrices always have a complete set of real orthogonal eigenvectors (Spectral Theorem).
- Similarly AA^T is always real symmetric PSD.

Therefore, V and U always exist, and hence **every real matrix** $A \in \mathbb{R}^{m \times n}$ **has an SVD**.

Geometric Interpretation

Any linear transformation $\mathbb{R}^n \rightarrow \mathbb{R}^m$ can be decomposed into three steps: (1) rotate/reflect in the input space (by V^T); (2) scale along axes (by Σ); (3) rotate/reflect in the output space (by U).

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem**
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

SVD as a Sum of Rank-1 Matrices

Outer Product Expansion

From $A = U\Sigma V^T = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$:

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots + \sigma_k \mathbf{u}_k \mathbf{v}_k^T$$

Each term $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ is a **rank-1 matrix** (outer product of two vectors). They are added in order of decreasing importance (since $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \geq 0$). Including all k terms recovers A exactly. **Dropping the last $k - r$ terms** gives a rank- r approximation.

Storage Comparison

Original A : $m \times n$ entries. Rank- r approximation: need to store r triples $(\mathbf{u}_i, \sigma_i, \mathbf{v}_i)$: $(m + n + 1) \times r$ values. Compression ratio $\approx \frac{mn}{(m + n)r}$. For $m = 400$, $n = 10,000$, $r = 50$: original 4×10^6 , compressed $\approx 5 \times 10^5$ — $8\times$ reduction.

The Eckart–Young–Mirsky Theorem

Theorem 1 (Best Low-Rank Approximation)

Let $A \in \mathbb{R}^{m \times n}$ with SVD $A = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$. The **best rank- r approximation** of A in the Frobenius norm is:

$$A_r = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

That is, A_r solves:

$$\min_{B: \text{rank}(B) \leq r} \|A - B\|_F^2$$

and the minimum error is:

$$\|A - A_r\|_F^2 = \sum_{i=r+1}^k \sigma_i^2$$

Why This Is Deep

The theorem says: if you must approximate A by a rank- r matrix and you want to minimise the total squared entry-wise error, the answer is exactly the first r terms of the SVD expansion. **The singular values directly quantify reconstruction quality.**

Frobenius Norm: What We Are Minimising

Definition

The Frobenius norm of a matrix $A \in \mathbb{R}^{m \times n}$:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(A^\top A)}$$

It is the *square root of the sum of squared entries* — the natural generalisation of the Euclidean norm to matrices. $\|A - B\|_F^2 = \sum_{i,j} (a_{ij} - b_{ij})^2$ measures entry-wise squared differences.

Connection to Singular Values

Since $\text{tr}(A^\top A) = \text{tr}(V \Sigma^2 V^\top) = \text{tr}(\Sigma^2) = \sum_i \sigma_i^2$:

$$\|A\|_F^2 = \sum_{i=1}^k \sigma_i^2$$

So the singular values fully determine the Frobenius norm. Dropping the $r+1, \dots, k$ terms loses $\sum_{i=r+1}^k \sigma_i^2$ — the “energy” in those directions.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection**
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

PCA via SVD of the Centred Data Matrix

Direct SVD of X_c

Apply SVD to the centred data matrix $X_c \in \mathbb{R}^{m \times n}$:

$$X_c = U \Sigma V^T$$

$V \in \mathbb{R}^{n \times n}$: right singular vectors.

Now compute $X_c^T X_c$:

$$X_c^T X_c = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma^T U^T U \Sigma V^T = V \Sigma^2 V^T$$

Comparing with the EVD $\Sigma_{\text{cov}} = P \Lambda P^T$:

- **Principal directions** $P = V$ (right singular vectors of X_c)
- **Eigenvalues** $\Lambda = \frac{1}{m} \Sigma^2$, i.e. $\lambda_i = \sigma_i^2 / m$

Therefore, **PCA of X_c is equivalent to the SVD of X_c** . One can run PCA by directly SVD-decomposing the data matrix — often more numerically stable than computing $X_c^T X_c$ first.

Row Space, Column Space, Rank, and Null Space

Fundamental Subspaces of $A \in \mathbb{R}^{m \times n}$

- **Column space** (range of A): $\text{col}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$. Dimension = $\text{rank}(A)$.
- **Row space**: $\text{col}(A^\top) \subseteq \mathbb{R}^n$. Dimension = $\text{rank}(A)$.
- **Null space**: $\text{null}(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\} \subseteq \mathbb{R}^n$. Dimension = $n - \text{rank}(A)$ (Rank–Nullity Theorem).
- **Left null space**: $\text{null}(A^\top) \subseteq \mathbb{R}^m$.

SVD Reveals the Subspaces

The right singular vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ span the **row space** of A . The remaining $\mathbf{v}_{k+1}, \dots, \mathbf{v}_n$ span the **null space**. The left singular vectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ span the **column space**. The remaining $\mathbf{u}_{k+1}, \dots, \mathbf{u}_m$ span the **left null space**.

Consequence: only the k dimensional subspace of $\mathbb{R}^n$ in the row space can generate a non-zero output; null-space vectors are annihilated. So we only need $k = \text{rank}(A)$ basis vectors to fully represent what A does.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition**
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics

Eigenfaces: Setup

The Data

m training face images, each of size 100×100 pixels $\Rightarrow n = 10,000$ features per image.
Data matrix $X \in \mathbb{R}^{m \times 10,000}$. After centring: X_c with zero-mean columns.

Computing Eigenfaces via the Gram Matrix Trick

Direct EVD of Σ is infeasible ($10,000 \times 10,000$). Instead:

- 1 Compute $G = \frac{1}{m} X_c X_c^\top \in \mathbb{R}^{m \times m}$
- 2 Solve $G \mathbf{v}_i = \lambda_i \mathbf{v}_i$ (cheap: $m \times m$)
- 3 Recover eigenfaces: $\mathbf{p}_i = \frac{X_c^\top \mathbf{v}_i}{\sqrt{m \lambda_i}} \in \mathbb{R}^{10,000}$
- 4 Reshape each $\mathbf{p}_i$ to 100×100 for visualisation — these are the **eigenfaces**
- 5 Keep top- k eigenfaces (e.g. $k = 50$ – 100)

Properties of Eigenfaces

- They form an orthonormal basis for the space of faces in the dataset
- Any face $\approx \boldsymbol{\mu} + \sum_{i=1}^k z_i \mathbf{p}_i$
- Store $\mathbf{p}_1, \dots, \mathbf{p}_k$ once; then only k scalars z_i per new image

Eigenfaces: Compression and Recognition

Storage Analysis

Original dataset: $m \times n = m \times 10,000$ numbers. After PCA with k components:

$$\underbrace{k \times n}_{\text{eigenfaces } P_k} + \underbrace{n}_{\text{mean face } \mu} + \underbrace{m \times k}_{\text{scores } Z} \text{ numbers total}$$

Numerical example: $m = 400$, $n = 10,000$, $k = 50$:

- Original: $400 \times 10,000 = 4,000,000$ values
- Compressed: $50 \times 10,000 + 10,000 + 400 \times 50 = 510,000$ values
- Compression factor: $\approx 7.8\times$

Generalisation and Limitations

The eigenfaces basis is learned from training data. Any new face can be projected onto this basis and (approximately) reconstructed. If training data is diverse (many demographics, lighting conditions), the basis generalises well. If training data is narrow (only one demographic), the basis will reconstruct out-of-distribution faces poorly. Same principle as any machine learning model: quality of the learned representation depends on diversity of training data.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections**
- 10 Summary of Basics

Summary: The Big Picture

EVD (Square Matrices Only)

$$A = V\Lambda V^{-1}$$

Symmetric: $A = V\Lambda V^T$

Requires: square; sufficient independent eigenvectors

Gives: eigenvalues λ_i , eigenvectors $\mathbf{v}_i$

$$A\mathbf{v}_i = \lambda_i\mathbf{v}_i$$

SVD (Any Matrix)

$$A = U\Sigma V^T$$

U : left singular vectors (eigenvectors of AA^T)

V : right singular vectors (eigenvectors of $A^T A$)

$\sigma_i = \sqrt{\lambda_i}$: singular values

Always exists; no shape restriction

PCA (via EVD or SVD)

Goal: Find low-dimensional representation

Tool: EVD of $\Sigma = \frac{1}{m}X_c^T X_c$

Shortcut: SVD of X_c directly

Principal directions = eigenvectors of Σ
= right singular vectors of X_c

Variance along $\mathbf{p}_i = \lambda_i$

Gram Matrix Trick

When $m \ll n$:

EVD of $G = \frac{1}{m}X_c X_c^T$ ($m \times m$)

then $\mathbf{p}_i = \frac{X_c^T \mathbf{v}_i}{\sqrt{m\lambda_i}}$

Cost: $O(m^3)$ instead of $O(n^3)$

“SVD always exists. EVD needs square matrices. PCA is EVD of the covariance matrix — or equivalently, SVD of the data matrix.”

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 SVD and PCA: The Complete Connection
- 8 Eigenfaces: PCA Applied to Face Recognition
- 9 Summary: SVD, PCA, and Their Connections
- 10 Summary of Basics**

Cauchy–Schwarz Inequality

Statement

For any two vectors $\mathbf{u}, \mathbf{v}$ in an inner product space:

$$|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\| \|\mathbf{v}\|$$

In $\mathbb{R}^n$ (dot product form):

$$\left(\sum_{i=1}^n u_i v_i \right)^2 \leq \left(\sum_{i=1}^n u_i^2 \right) \left(\sum_{i=1}^n v_i^2 \right)$$

Proof via Cosine Formula

By definition of the inner product in $\mathbb{R}^n$:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

Since $|\cos \theta| \leq 1$ for all $\theta \in [0, \pi]$:

$$|\langle \mathbf{u}, \mathbf{v} \rangle| = \|\mathbf{u}\| \|\mathbf{v}\| |\cos \theta| \leq \|\mathbf{u}\| \|\mathbf{v}\|$$

Equality holds iff $|\cos \theta| = 1$, i.e. $\theta = 0$ or π , i.e. $\mathbf{u} = \lambda \mathbf{v}$ for some scalar λ (vectors are linearly dependent).

Connection to PCA

The projection coefficient $\alpha_i = \mathbf{z}^\top \mathbf{p}_i$ satisfies $|\alpha_i| \leq \|\mathbf{z}\| \|\mathbf{p}_i\| = \|\mathbf{z}\|$ (since $\|\mathbf{p}_i\| = 1$). This bounds how much variance a single principal component can capture. The correlation coefficient $\rho_{xy} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}$ satisfies $|\rho_{xy}| \leq 1$ precisely because of Cauchy–Schwarz applied to the vectors of centred observations.

Numerical Example: EVD of AA^T

Setup

Let $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$. Compute:

$$AA^T = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 9 & 12 \\ 12 & 41 \end{bmatrix}$$

Finding Eigenvalues: Characteristic Polynomial

$$\det(AA^T - \lambda I) = \det \begin{bmatrix} 9-\lambda & 12 \\ 12 & 41-\lambda \end{bmatrix} = (9-\lambda)(41-\lambda) - 144 = 0$$

Expanding: $\lambda^2 - 50\lambda + 369 - 144 = \lambda^2 - 50\lambda + 225 = 0$. Quadratic formula:

$$\lambda = \frac{50 \pm \sqrt{2500 - 900}}{2} = \frac{50 \pm 40}{2}.$$

$$\lambda_1 = 45, \quad \lambda_2 = 5$$

Singular values of A : $\sigma_1 = \sqrt{45} = 3\sqrt{5}$, $\sigma_2 = \sqrt{5}$.

Finding Eigenvectors

For $\lambda_1 = 45$: $(AA^T - 45I)\mathbf{v} = \mathbf{0}$ gives $-36x + 12y = 0 \Rightarrow y = 3x$. Normalised:

$$\mathbf{u}_1 = \frac{1}{\sqrt{10}}[1, 3]^T.$$

For $\lambda_2 = 5$: $4x + 12y = 0 \Rightarrow x = -3y$. Normalised: $\mathbf{u}_2 = \frac{1}{\sqrt{10}}[-3, 1]^T$.

Check orthogonality: $\mathbf{u}_1^T \mathbf{u}_2 = \frac{1}{10}(1 \cdot (-3) + 3 \cdot 1) = 0$. ✓ (Guaranteed since AA^T is symmetric.)

Condition Number and Why SVD Is Numerically Superior

Definition: Condition Number

For a matrix X with singular values $\sigma_1 \geq \sigma_2 \geq \dots$, the **condition number** is:

$$\kappa(X) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

It measures how much the matrix can *amplify* relative errors in the input. Large κ means the matrix is nearly singular and numerical errors are magnified.

Why Forming $X^\top X$ Squares the Condition Number

Let $X = U\Sigma V^\top$ (SVD). Then:

$$X^\top X = V\Sigma^\top U^\top U\Sigma V^\top = V\Sigma^2 V^\top$$

The eigenvalues of $X^\top X$ are σ_i^2 (squares of singular values of X). Therefore:

$$\kappa(X^\top X) = \frac{\sigma_{\max}^2}{\sigma_{\min}^2} = \left(\frac{\sigma_{\max}}{\sigma_{\min}}\right)^2 = \kappa(X)^2$$

Example: If $\kappa(X) = 10^3$, then $\kappa(X^\top X) = 10^6$. If $\sigma_{\min} = 10^{-8}$, then $\sigma_{\min}^2 = 10^{-16}$, which is at the threshold of 64-bit floating point machine precision ($\approx 10^{-16}$) — the computer may treat it as zero, completely destroying that direction's information.

Practical Conclusion

Direct SVD algorithms (Golub–Kahan bidiagonalisation, QR iterations with Householder reflections) compute U, Σ, V from X *without ever forming* $X^\top X$. They work with the original singular values σ_i rather than their squares, keeping numerical errors minimal. This is why `sklearn.decomposition.PCA` uses SVD internally.

Theory vs. Practice: Finding V in SVD

What Theory Says

From $A = U\Sigma V^\top$, compute:

$$A^\top A = V\Sigma^2 V^\top$$

So V consists of the eigenvectors of $A^\top A$, and U consists of the eigenvectors of $AA^\top$. The singular values satisfy $\sigma_i = \sqrt{\lambda_i}$. This is the mathematical *derivation* that connects SVD to EVD.

What Practice Does

Modern SVD algorithms **do not form** $A^\top A$ or $AA^\top$. They work directly on A using orthogonal transformations:

- 1 **Bidiagonalisation** (Householder reflections): reduce A to a bidiagonal matrix B such that $A = U_1 B V_1^\top$ with orthogonal U_1, V_1 .
- 2 **QR iteration** on B : compute SVD of B numerically without forming $B^\top B$.

At no step are eigenvalues of $A^\top A$ computed. The singular values are found directly as σ_i , not via $\sqrt{\lambda_i(A^\top A)}$, so the squaring step and its numerical damage are avoided entirely.

The Key Distinction

Theory: V are eigenvectors of $A^\top A$ — useful for understanding and deriving.

Practice: SVD algorithms compute V from A directly, bypassing $A^\top A$ to preserve numerical precision.

Bessel's Correction: Why Divide by $n - 1$?

The Problem with Dividing by n

Given i.i.d. samples $X_1, \dots, X_n$ with true variance σ^2 , define the naive sample variance: $S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$. We will show $\mathbb{E}[S_n^2] = \frac{n-1}{n} \sigma^2 \neq \sigma^2$ — it is **biased downward**.

Key Identity

Expand by adding and subtracting μ :

$$\begin{aligned}\sum_{i=1}^n (X_i - \bar{X})^2 &= \sum_{i=1}^n [(X_i - \mu) - (\bar{X} - \mu)]^2 \\ &= \sum_{i=1}^n (X_i - \mu)^2 - 2(\bar{X} - \mu) \sum_{i=1}^n (X_i - \mu) + n(\bar{X} - \mu)^2 \\ &= \sum_{i=1}^n (X_i - \mu)^2 - n(\bar{X} - \mu)^2\end{aligned}$$

where we used $\sum_{i=1}^n (X_i - \mu) = n\bar{X} - n\mu = n(\bar{X} - \mu)$.

Bessel's Correction: Full Derivation

Taking Expectations

Apply $\mathbb{E}[\cdot]$ (using linearity) to the identity $\sum (X_i - \bar{X})^2 = \sum (X_i - \mu)^2 - n(\bar{X} - \mu)^2$:

$$\mathbb{E}\left[\sum_{i=1}^n (X_i - \bar{X})^2\right] = \mathbb{E}\left[\sum_{i=1}^n (X_i - \mu)^2\right] - n\mathbb{E}[(\bar{X} - \mu)^2]$$

First term: $\mathbb{E}[(X_i - \mu)^2] = \text{Var}(X_i) = \sigma^2$ for each i , so $\mathbb{E}[\sum_{i=1}^n (X_i - \mu)^2] = n\sigma^2$.

Second term: $\mathbb{E}[(\bar{X} - \mu)^2] = \text{Var}(\bar{X})$. Since X_i are i.i.d.:

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}$$

So $n\mathbb{E}[(\bar{X} - \mu)^2] = n \cdot \frac{\sigma^2}{n} = \sigma^2$.

Combining:

$$\mathbb{E}\left[\sum_{i=1}^n (X_i - \bar{X})^2\right] = n\sigma^2 - \sigma^2 = (n-1)\sigma^2$$

Bessel's Correction: Conclusion and Degrees of Freedom

Biased vs. Unbiased Estimator

From the derivation: $\mathbb{E}[\sum_{i=1}^n (X_i - \bar{X})^2] = (n-1)\sigma^2$.

Dividing by n gives a **biased** estimator:

$$\mathbb{E}[S_n^2] = \mathbb{E}\left[\frac{1}{n} \sum (X_i - \bar{X})^2\right] = \frac{1}{n}(n-1)\sigma^2 = \frac{n-1}{n}\sigma^2 \neq \sigma^2$$

Dividing by $n-1$ gives an **unbiased** estimator:

$$\mathbb{E}[S^2] = \mathbb{E}\left[\frac{1}{n-1} \sum (X_i - \bar{X})^2\right] = \frac{1}{n-1}(n-1)\sigma^2 = \sigma^2 \quad \checkmark$$

Why $n-1$? The Degrees of Freedom Argument

The deviations $d_i = X_i - \bar{X}$ satisfy the constraint $\sum_{i=1}^n d_i = 0$ (deviations always sum to zero by definition of the mean). This means once $n-1$ deviations are known, the n -th is completely determined. Only $n-1$ deviations are *free to vary* — there are $n-1$ **degrees of freedom**. Intuitively: we used one degree of freedom to *estimate* μ by $\bar{X}$ from the same data, leaving $n-1$ independent pieces of information for the variance estimate.

Connection to PCA / Covariance Matrix

In PCA with zero-mean data, we use $\Sigma = \frac{1}{m} X_c^\top X_c$. The factor m is a convention common in machine learning (population variance, not sample variance). If using sample covariance (statistics convention), divide by $m-1$. For large m , the difference is negligible and does not affect which eigenvectors are found — only the scale of eigenvalues.

Thank You

Questions?

Summary of Basics — Cauchy–Schwarz Inequality

Statement (Inner-Product Form)

For any vectors $\mathbf{u}, \mathbf{v}$ in an inner-product space:

$$|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\| \|\mathbf{v}\|$$

In $\mathbb{R}^n$ with the standard dot product:

$$\left(\sum_{i=1}^n u_i v_i \right)^2 \leq \left(\sum_{i=1}^n u_i^2 \right) \left(\sum_{i=1}^n v_i^2 \right)$$

Proof via the Cosine Formula

Recall the projection of $\mathbf{u}$ onto $\mathbf{v}$:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

where θ is the angle between them. Since $|\cos \theta| \leq 1$ for all θ :

$$|\langle \mathbf{u}, \mathbf{v} \rangle| = \|\mathbf{u}\| \|\mathbf{v}\| |\cos \theta| \leq \|\mathbf{u}\| \|\mathbf{v}\|$$

Equality holds $\Leftrightarrow |\cos \theta| = 1 \Leftrightarrow \theta \in \{0^\circ, 180^\circ\} \Leftrightarrow \mathbf{u} = \lambda \mathbf{v}$ for some scalar λ (vectors are *linearly dependent*).

Numerical Check: $\mathbf{u} = (1, 2)$, $\mathbf{v} = (3, 4)$

$\langle \mathbf{u}, \mathbf{v} \rangle = 11$; $\|\mathbf{u}\| \|\mathbf{v}\| = \sqrt{5} \cdot 5 = 5\sqrt{5} \approx 11.18$. Indeed $11 \leq 11.18$. ✓

Why It Appears in PCA

The correlation coefficient $\rho_{xy} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}$ satisfies $|\rho_{xy}| \leq 1$ precisely because of

Cauchy–Schwarz: $|\text{Cov}(x, y)| \leq \sqrt{\text{Var}(x) \text{Var}(y)}$. This guarantees the covariance matrix is PSD and that correlation is a bounded, well-defined measure.

Summary of Basics — Condition Number

Definition

For a matrix X with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, the **condition number** is:

$$\kappa(X) = \frac{\sigma_{\max}}{\sigma_{\min}} = \frac{\sigma_1}{\sigma_r}$$

It measures the *ratio of maximum to minimum stretching* performed by X . A large κ means the matrix nearly collapses some directions: small input perturbations in those directions become large output errors.

Why $X^\top X$ Squares the Condition Number

Let $X = U\Sigma V^\top$ (SVD). Then $X^\top X = V\Sigma^2 V^\top$, whose eigenvalues are $\sigma_1^2 \geq \dots \geq \sigma_r^2$. Therefore:

$$\kappa(X^\top X) = \frac{\sigma_1^2}{\sigma_r^2} = \left(\frac{\sigma_1}{\sigma_r}\right)^2 = \kappa(X)^2$$

Forming $X^\top X$ squares the condition number. Example: if $\kappa(X) = 10^3$, then $\kappa(X^\top X) = 10^6$. If $\sigma_{\min}(X) = 10^{-8}$, then $\sigma_{\min}(X^\top X) = 10^{-16}$, which is at machine-epsilon: the computer may treat this eigenvalue as zero, destroying accuracy.

Summary of Basics — Why PCA Uses SVD Directly

Two Routes to the Principal Directions V

Route 1 (Covariance / EVD):

$$X \xrightarrow{\text{form}} X^T X \xrightarrow{\text{EVD}} V$$

Eigenvalues of $X^T X$ are $\lambda_i = \sigma_i^2$. Condition number goes from $\kappa(X)$ to $\kappa(X)^2$.

Route 2 (Direct SVD):

$$X \xrightarrow{\text{SVD}} U, \Sigma, V$$

Modern algorithms (Golub–Kahan bidiagonalisation + QR iteration) apply orthogonal transformations (Householder reflections) *directly* to X — they never form $X^T X$. They compute σ_i without squaring them, so numerical precision is preserved.

Worked Condition Number Example

Suppose $\sigma_{\max}(X) = 10^4$, $\sigma_{\min}(X) = 10^{-4}$.

Approach	Extremal values seen	κ
Direct SVD on X	$10^4, 10^{-4}$	10^8
EVD on $X^T X$	$10^8, 10^{-8}$	10^{16}

At $\kappa = 10^{16}$ the ratio exceeds double-precision machine epsilon ($\approx 10^{-16}$): the smallest eigenvalue is indistinguishable from zero numerically. That is why `sklearn.decomposition.PCA` uses SVD internally.

Summary of Basics — Computing Eigenvalues of AA^T

Setup

Let $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$. Compute AA^T :

$$AA^T = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 9 & 12 \\ 12 & 41 \end{bmatrix}$$

Finding Eigenvalues via the Characteristic Polynomial

Solve $\det(AA^T - \lambda I) = 0$:

$$\det \begin{bmatrix} 9 - \lambda & 12 \\ 12 & 41 - \lambda \end{bmatrix} = (9 - \lambda)(41 - \lambda) - 144 = 0$$

Expand: $\lambda^2 - 50\lambda + 369 - 144 = 0$, i.e. $\lambda^2 - 50\lambda + 225 = 0$. By the quadratic formula:

$$\lambda = \frac{50 \pm \sqrt{2500 - 900}}{2} = \frac{50 \pm 40}{2} \Rightarrow \lambda_1 = 45, \quad \lambda_2 = 5$$

Singular values of A : $\sigma_1 = \sqrt{45} = 3\sqrt{5}$, $\sigma_2 = \sqrt{5}$. Condition number: $\kappa(A) = \sqrt{45/5} = 3$.

Summary of Basics — Eigenvectors of AA^T (Worked)

Eigenvector for $\lambda_1 = 45$

$$(AA^T - 45I)\mathbf{u} = \mathbf{0}:$$

$$\begin{bmatrix} -36 & 12 \\ 12 & -4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0} \Rightarrow -36x + 12y = 0 \Rightarrow y = 3x$$

Eigenvector (unnormalised): $[1, 3]^T$. Normalised: $\mathbf{u}_1 = \frac{1}{\sqrt{10}}[1, 3]^T$.

Eigenvector for $\lambda_2 = 5$

$$(AA^T - 5I)\mathbf{u} = \mathbf{0}:$$

$$\begin{bmatrix} 4 & 12 \\ 12 & 36 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0} \Rightarrow 4x + 12y = 0 \Rightarrow x = -3y$$

Eigenvector (unnormalised): $[-3, 1]^T$. Normalised: $\mathbf{u}_2 = \frac{1}{\sqrt{10}}[-3, 1]^T$.

Orthogonality Check

$\mathbf{u}_1^T \mathbf{u}_2 = \frac{1}{10}(1 \cdot (-3) + 3 \cdot 1) = 0$. ✓ (As guaranteed by the Spectral Theorem: eigenvectors of a real symmetric matrix corresponding to *distinct* eigenvalues are orthogonal.)

Summary of Basics — Bessel's Correction: The Key Lemma

Claim

$$E \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] = (n-1)\sigma^2$$

where $X_1, \dots, X_n \stackrel{\text{i.i.d.}}{\sim}$ distribution with mean μ and variance σ^2 , and $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$.

Proof: Algebraic Decomposition

Write $X_i - \bar{X} = (X_i - \mu) - (\bar{X} - \mu)$. Then:

$$\begin{aligned} \sum_{i=1}^n (X_i - \bar{X})^2 &= \sum_{i=1}^n (X_i - \mu)^2 - 2(\bar{X} - \mu) \underbrace{\sum_{i=1}^n (X_i - \mu)}_{n(\bar{X} - \mu)} + n(\bar{X} - \mu)^2 \\ &= \sum_{i=1}^n (X_i - \mu)^2 - n(\bar{X} - \mu)^2 \end{aligned}$$

Take expectation:

$$\begin{aligned} E \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] &= E \left[\sum_{i=1}^n (X_i - \mu)^2 \right] - nE[(\bar{X} - \mu)^2] \\ &= n\sigma^2 - n \cdot \frac{\sigma^2}{n} = n\sigma^2 - \sigma^2 = (n-1)\sigma^2. \end{aligned} \quad \text{(using } E[(X_i - \mu)^2] = \sigma^2 \text{ and } \text{Var}(\bar{X}) = \sigma^2/n \text{)}$$

Summary of Basics — Bessel's Correction: Bias and Unbiasedness

Why Dividing by n is Biased

Define $S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$. By the lemma and linearity of expectation ($E[cY] = cE[Y]$):

$$E[S_n^2] = \frac{1}{n} E \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] = \frac{1}{n} (n-1) \sigma^2 = \frac{n-1}{n} \sigma^2 \neq \sigma^2$$

The estimator systematically *underestimates* σ^2 by a factor $\frac{n-1}{n}$.

Bessel's Correction: Divide by $n-1$

Define $S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$. Then:

$$E[S^2] = \frac{1}{n-1} (n-1) \sigma^2 = \sigma^2$$

S^2 is an **unbiased estimator** of σ^2 .

Degrees of Freedom: Why $n-1$?

The n deviations $d_i = X_i - \bar{X}$ satisfy $\sum_{i=1}^n d_i = 0$ (since $\sum X_i = n\bar{X}$). This single linear constraint reduces the dimension of the space in which $(d_1, \dots, d_n)$ lives from n to $n-1$. Knowing any $n-1$ deviations determines the n -th: $d_n = -\sum_{i=1}^{n-1} d_i$. Hence there are $n-1$ **degrees of freedom** — and we divide by $n-1$.

Connection to PCA

In PCA we use $\Sigma = \frac{1}{m} X_c^T X_c$ (divide by m , not $m-1$) — a population-variance convention common in ML. Eigenvectors are identical either way; only eigenvalue *scale* differs by $\frac{m}{m-1}$, which is negligible for large m .